GPU Programming

Created	@March 26, 2025 4:48 PM
Class	High Performance Computing

Computing with GPUs

- “data parallel” algorithms are great on gpus

Data-Parallel Execution & the SPMD Model

- **SPMD (Single Program, Multiple Data)**: You write one kernel function; the runtime launches thousands of “logical” threads, each executing the same code but operating on different data elements
- **Grid → Blocks → Threads**:
  - a grid is a 1D/2D/3D array of blocks
  - each block is itself a 1D/2D/3D array of threads
  - thread indices (`blockIdx, threadIdx`) let each thread computr a unique data offset

```
__global__ void saxpy(float a, float *x, float*y, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) y[i] = a*x[i] + y[i];
}

//host launch:
int threads = 256;
int blocks = (n + threads - 1)/threads;
saxpy<<<blocks, threads>>>(a, x_d, y_d, n);
```

Thread Blocks and Warp Scheduling

- **Warps**: a block is sliced into 32 thread warps (hardware scheduling units).
  - all threads in a warp must execute the same instruction; divergent branches serialize within the warp
  - basic unit of executyion on an NVIDIA GPU

    - its a group of 32 threads that execute the same instruction at the same time (lock step)
    - when you launch a block of 256 threads, the hardware divides it into 256/32 = 8 warps
    - why 32? Its chosen to match the width of the GPU's vector pipelines and memory-transaction hardware
    - **Divergence**: If threads within a war p take different branches, the warp serializes those paths —one path at a time—so divergent code hurts performance
- **Block-local cooperation**:
    - **Shared memory** (on-SM) is a user-managed cache. Threads in the same block can exchange data via shared array and sync with `__syncthreads()`
    - **Atomics** on shared or global memory let threads coordinate counters, reductions, etc
- **SM (Streaming Multiprocessor)**:
    - an SM is like a "core" on a GPU
    - it contains many CUDA corses, ALU, special-function units, load/store units, register file, and shared-memory banks
    - it schedules and issues instruction for multiple warps, toggling between them to hide latency
    - when you launch a kernel, the grid's blocks are distributed across the SMs. Each SM can hold several active blocks (and thus many warps) simultaneously up to the hardware limits on registers and shared memory

The CUDA Memory Hierarchy

Scope	Lifetime	Latency	Typical Use
Registers	Per thread	~1 cycle	Private variables, temporaries
Shared memory	Per block	~10 cycles	Staging data, tiling, reductions
L1/L2 caches	Hardware	~30-100 cycles	Implicit, for data reuse
Global memory	Persistent	~300-800 cycles	Large arrays, input/output
Constant/Texture	Read-only	Cached	Broadcast constants, 2D data

- Effective Bandwidth

$$BW_{eff} = \frac{\text{bytes read} + \text{bytes written}}{\text{kernel time (s)}} \quad \text{[GB/s]}$$

Compare this to GPU's peak to see if you're memory bound

Memory Access Patterns & Coalescing

- **Coalesced loads/stores**: Consecutive threads in a warp should access consecutive addresses
- **Strided or scattered** accesses force multiple memory transactions, killing throughput
- **Tiling/shared-memory blocking**:

1. load a tile of global data into shared memory(coalesced)
2. sync threads
3. perform computations reusing the tile
4. write results back

Occupancy & Launch Configuration

- **Occupancy:** Ratio of active warps per SM to the hardware maximum
- **Trade-offs**:
  - **More threads/block** → higher occupancy → better ability to hide latency, but ...
  - **Larger shared-memory or register usage per block** → fewer blocks can reside on an SM
- **Tools** `cudaOccupancyMaxPotentialBlockSize` API, NVIDIA Nsight Compute

Synchronization & Dependencies

- within a block: use `__syncthreads()` to enforce that all threads reach a barrier before proceeding
- Across blocks: no implicit barrier—must split into separate kernel launches if you ned a global sync

Profiling & Performance Tuning

- **nvprof/ Nsight Systems: identify wheter your limited by compute (low ALU utilization) or memory (poor bandwidth)**
- Metrics to watch:
  - achieved occupancy
  - warp-execution efficiency
  - DRAM throughput vs. peak

Example:

Tiled matrix-multiply kernel sketch:

```
const int TILE = 16;

__global void matmul_tiled(float *A, float*B, float *C, int N) {
   __shared__ float As[TILE][TILE], Bs[TILE][TILE];
   int row = blockIdx.y * TILE + threadIdx.y;
   int col = blockIdx.x * TILE + theadIdx.x
   float acc = 0.0f;
   for (int t = 0; t < (N + TILE - 1)/TILE ++t) {
```

```
    //coalexced load into shared
    As[threadIdx.y][threadIdx.x] = A[row*N + t*TILE + threadIdx.x];
    Bs[threadIdx.y][threadIdx.x] = B[(t*TILE + threadIdx.y)*N + col];
    __syncthreads();

    for (int k = 0; k < TILE; ++k)
       acc += As[threadIdx.y][k] * Bs[k][threadIdx.x];
       __synchthreads();
    }
    C[row*N + col] = acc;
}
```

- threads load contiguous chunks → coalesced
- shared-memory reuse reduces globa-memory traffic
- `__synchthreads()` enforces correct staging

Execution Model

- host + device
- program starts on the cpu
- data on disk, will go to main memory
- a CUDA kernel is executed as a grid(array) of threads
  - all threads in a grid run the same kernel code
  - single program multiple data (SPMD model)
  - each thread has a unique index that it uses to compute memory addresses and make control decisions

- each CUDA kernel
  - is executed by a grid
  - a 3d array of thread blocks, with are 3d arrays of threads
  - each block has an id

Thread Blocks: Scalable Cooperations

- divide thread array into multiple blocks
  - threads within a block cooperate via shared memory, atomic operations, and barrier synchronization
  - threads in different blocks cooperate less
- each thread uses indices to decide what data to work on
  - block idx, threadidx

GPU CUDA Execution Model

- all threads in a block execute the same kernal
- programmar declares block geometry
  - each block is executed as 32 thread warps
    - an implementation decision not part of the CUDA programming model
    - warps are divided based on their linearized thread index
    - warps are shceduling units in SM

GPU Memory Model

Bandwidth performance

- rate at which data can be transfered
- gpus ahve a theoretical bandwidhgt
- to undersand hwo good the performance of you rprogram are compared to the peak performance you can compute the effective bandwidth
- $BW_{effective} = (R_B + W_B)/(t * 10^9)\backslash$
- measured in GB/s
- SM implementes zero overhead warp scheduleing
- warps whose next instruction has its operandes ready for consumption are eligible for execution
- eligible warps are selectedc for execution on a prioritized scheduling policy
- all threads in a warp execute the same instruction when selected

Memory Tiling

- each thread can read write per thread registers
- readwrite per block shared memory
- read/write per grid global memory
- read only per grid constand memory

How to Determine how many blocks to launch based on GPU's SM

- find SM count (ie 16)

- find how many block can be active per SM (ie 4)
- Grid size = blocksPerSM * numSMs (ie 64)
- you can allso cap by data size, so if you need to cover N elelments with threadsPerBlock threads each:
  - dataBlocks = (N + theadsPerBlock - 1) / threadsPerBlock
- then the number of blocks to launch = min(datablocks, gridSize)

How to determine threadsPerBlock

- every SM supports at most 1024 (sometimes 2048) threads per block
- blocks are internally scheduled in warps fo 32 threads, so block size should be a multipe of 32 to avoid under-fill3ed warps
- each block's threads consume registers and shared memory. If you ask for too many threads, you may exhaust one of those resources and reduce the number of blocks (and warps) that can reside on each SM

While you can write down

,

$$max_{T \in 32Z} = Tmin\left(\lfloor R_{SM}/(r_{thread}T)\rfloor \lfloor S_{SM}/s_{block}\rfloor,\ \lfloor W_{max}/(T/32)\rfloor,\ Bhw\right)/maxThreadsPerSM$$

in practice you brute-force over multiples of 32 (or use the Occupancy API) to find the best threads-per-block.